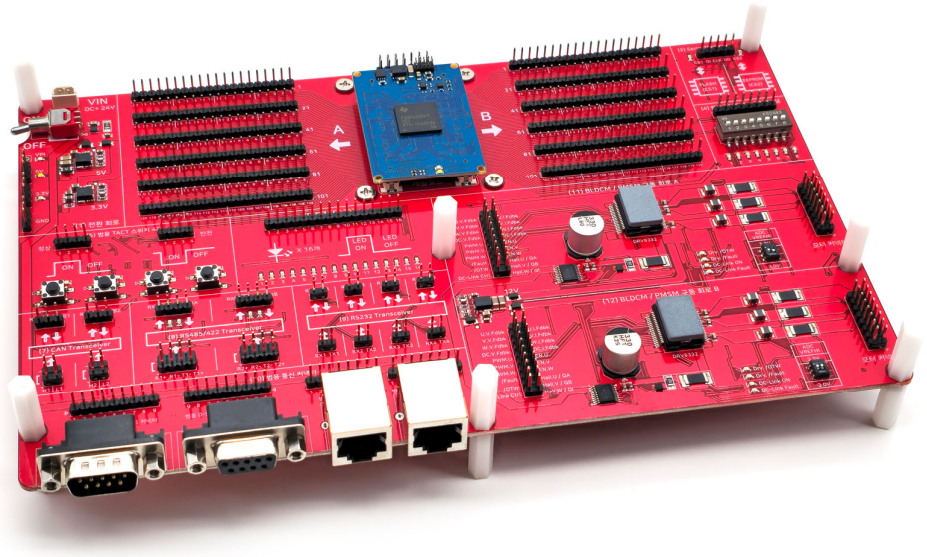
* 본 자료는 모터구동 학습용 예제로만 사용하십시오. 실제 시스템 적용으로 인한 문제에 대해서는 싱크웍스에서 책임지지 않습니다. 또한, 기술적인 지원도 할 수 없음을 알려드립니다.

* 본 예제 프로그램은 Texas Instruments(TI)사에서 TMS320F28X 프로세서 사용자들을 대상으로 공개한 Digital Motor Control Library를 기초로 만들어졌으며, SMC300 개발보드에 적합하도록 일부 수정되었습니다. (참조자료 : TI ControlSUITE에 포함된 DMC library)

* Revision 2026. 05. 21. (v1.21)

Hall-Effect Sensor를 이용한 3상 BLDCM 속도제어 (SMC300 개발보드용)

F28P65x
모듈 탑재
기준 안내



* Revision History

날 짜	내 용
25. 06. 16.	- V1.0 / 초판
25. 12. 23.	- V1.1 / SMC300 EVM V110 회로 설계에 맞춰 일부 수정
26. 05. 20.	- V1.2 / F28P559SJ9-Q1 모듈 사용에 맞춰 내용 수정
26. 05. 21.	- V1.21 / F28P65x 모듈 사용에 맞춰 내용 수정

* SMC300 개발보드 V100 → V110 회로설계 주요 변경 부분
  - 모터 구동 부, DC-LINK 전원 ON/OFF용 물리 스위치(Toggle) 제거
    (eFuse 소자에 의한 전자제어로 DC-LINK 전원 ON/OFF 제어 가능)

목차

1 실습 준비 4

2 주의 사항 4

3 실습 예제 설치 4

4 예제 실행 5

4.1 예제 실행 준비하기 6

4.2 실습 따라 하기 (BUILD LEVEL 1) 12

4.3 실습 따라 하기 (BUILD LEVEL 2) 21

4.4 실습 따라 하기 (BUILD LEVEL 3) 23

1 실습 준비

본 예제는 아래의 개발환경에서 테스트되었습니다.

>> **CCS 버전** : v12.8.1.00005

>> **Compiler 버전** : TI v25.11.0.LTS

>> **개발보드** : SMC300 모터제어 개발보드

>> **TMS320F28X 모듈** : TMS320F28P650DK9 모듈 / TMS320F28P659DK8-Q1 모듈

2 주의 사항

전력 변환이나 모터 구동 등의 실습은 고-전압 혹은 고-전류에 노출되어 있기 쉽습니다. 이는 부주의로 인한 사고 발생 시, 장비들을 손상시킬 뿐만 아니라 개발자들께도 위험을 줄 수 있습니다. 따라서 실습을 진행함에 있어 명시된 절차를 필히 지켜 주셔야 합니다. 또한 소비전류 수치의 확인 및 제한이 용이한 DC 전원공급기의 사용을 권고 드리며, 안전한 실습을 위해 예제 프로그램의 실행을 멈추기 전, 반드시 DC-LINK에 공급된 전원을 차단해주십시오. (SMC300 개발보드는 eFuse 스위치의 전자제어를 통해 컨트롤 전원과 별개로 모터에 공급되는 DC-LINK 전원을 차단할 수 있습니다. / SMC300 개발보드 매뉴얼을 참조하세요.)

3 실습 예제 설치

싱크웍스 홈페이지를 통해 다운로드 하신 예제는 실행형 설치 패키지로 제작되어 있으며, 압축해제 후 실행해주시면 지정된 경로에 자동으로 설치됩니다. (설치 파일 실행에 문제가 있다면, 관리자 권한으로 실행해보세요.)

예제의 설치 경로는 아래와 같습니다.

>> **C:\SyncWorks\SMC300_EVM_28P65x\c28x**

4 예제 실행

SMC300 개발보드용 'BLDCM_Sensored_Control' 예제는 헤더파일의 전-처리 정의구문을 통해 서로 다른 3개 프로그램이 선택되도록 되어 있습니다. 예제에 포함된 헤더파일에 정의된 빌드 레벨 옵션(BUILDLEVEL)을 원하는 값으로 설정하면 그에 맞는 코드들이 선택적으로 컴파일 됩니다.

11_BLDCM_Sensored_Control.h

```
#endif

#define LEVEL0          0       // [내부 테스트용] 홀-센서 신호 확인 (Open-loop)
//-----------------------------------------------------------------------------------------------
#define LEVEL1          1       // Open-loop 구동 단계 (Only CW rotation)
#define LEVEL2          2       // Open-loop 구동 단계 (CW / CCW rotation)
#define LEVEL3          3       // 속도제어 실행 (Closed-loop, CW / CCW rotation, SpeedRef : -0.9 ~ +0.9)
//-----------------------------------------------------------------------------------------------

#define BUILDLEVEL      LEVEL3

#define SYSCLK          200     // 200MHz (CPU 클럭 주파수)
#define TBCLK           200     // 200MHz (EPWM 모듈이 공급받는 클럭 주파수)
#define PWMFREQ         20      // 20kHz (PWM 출력 스위칭 주파수)
#define ISRFREQ         40      // 40kHz (주-인터럽트 서비스 루틴 실행 주기)
#define ALIGN_DUTY      0x028F  // For 24-v, 3000RPM BLDC motor (OZ-SYSTEMS)

#define RPM             3000    // 3000RPM (B60Q-H24051-D1 모터의 정격 RPM)
#define POLES           4       // 모터 극수 (4극, 2극쌍)
```

[Build Level 1]

Hall-Effect Sensor가 주는 정보로 해석한 정류(Commutation)시점에 따라 스위칭을 전환하여 모터를 회전시키며, 부하에 따라 가변되지 않는 일정한 Duty-ratio의 PWM 출력을 통해 Open-loop로 BLDC 모터를 구동합니다. 모터는 시계방향(CW)으로만 회전합니다.

[Build Level 2]

Hall-Effect Sensor가 주는 정보로 해석한 정류(Commutation)시점에 따라 스위칭을 전환하여 모터를 회전시키며, 부하에 따라 가변되지 않는 일정한 Duty-ratio의 PWM 출력을 통해 Open-loop로 BLDC 모터를 구동합니다. 모터가 사용자의 회전방향 설정에 따라 시계방향(CW) 혹은 시계반대방향(CCW)으로 회전합니다.

[Build Level 3]

Hall-Effect Sensor와 S/W 모듈을 통해 현재 모터의 속도을 알아내고, PI 제어를 통해 사용자가 지정한 속도지령을 추종하도록 한 Speed Closed-loop 구동단계 입니다. 속도지령 값 및 부호에 따라 모터가 시계방향(CW) 혹은 시계반대방향(CCW)으로 회전합니다.

이어지는 예제 따라 하기 안내 과정은 Code Composer Studio v12.x 와 TMS320F28P65x 모듈, SDS200i 절연 JTAG 에뮬레이터 사용을 전제로 작성되었습니다.

4.1 예제 실행 준비하기

1. 아래의 경로에서 Code Composer Studio(CCS)를 다운로드 한 후 PC에 설치합니다.

>> https://www.ti.com/tool/download/CCSTUDIO

2. Code Composer Studio를 실행합니다.

바탕화면 단축아이콘이나 프로그램 메뉴를 통해, PC에 설치된 CCS를 실행합니다.

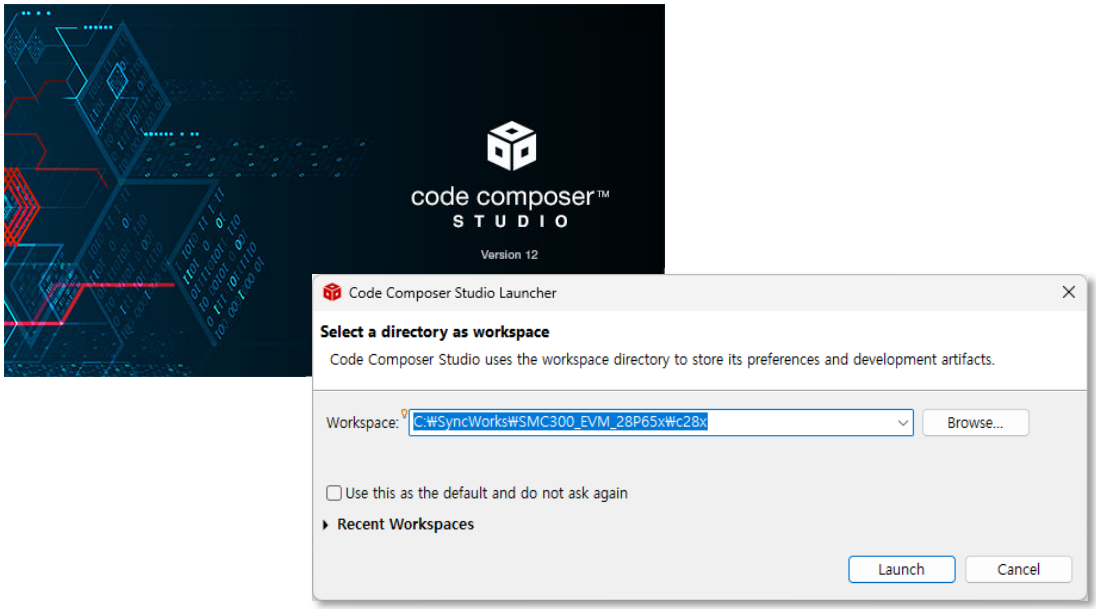
본 예제 프로그램은 CCS v12.8.1.00005 버전에서
제작 되었습니다. (컴파일러 : TI v25.11.0.LTS)

3. Workspace 경로를 선택합니다.

CCS Splash 이미지가 화면에 잠시 보이고, Workspace Launcher 창이 나타나게 됩니다. 이 창에서 Workspace를 예제의 설치경로 보다 한 단계 위 상위 경로로 지정합니다.

(Workspace 경로는 항상 예제 설치 경로보다 상위의 경로로 설정합니다. / 창 하단에 위치한 'Use this as default and do not ask again' 옵션은 선택하지 않습니다.)

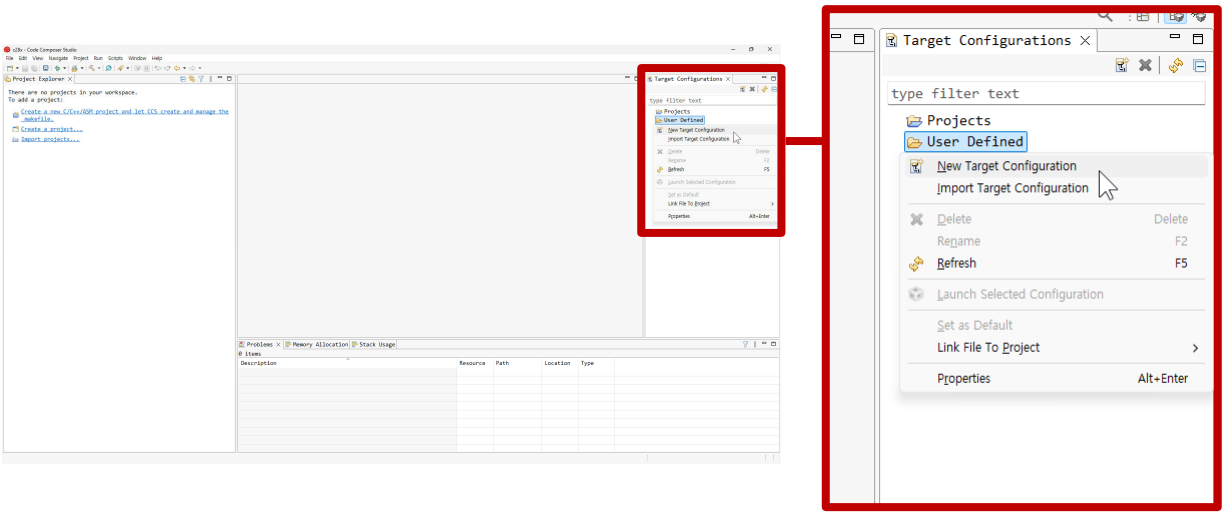
>> 예제 경로 : C:\SyncWorks\SMC300_EVM_28P65x\c28x\11_BLDCM_Sensored_Control
>> Workspace : **C:\SyncWorks\SMC300_EVM_28P65x\c28x**



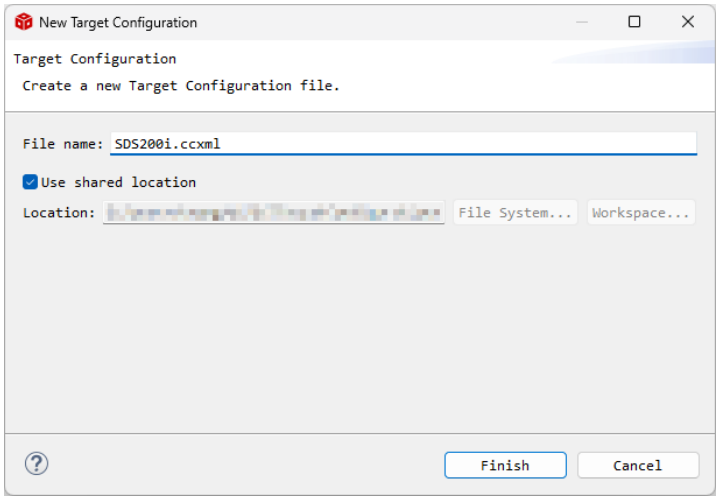
4. JTAG 에뮬레이터 종류 및 디버깅 대상(Target) 프로세서를 설정 합니다.

CCS 상단메뉴 View → Target Configuration을 선택합니다.

화면에 나타난 'Target Configurations' 창에서 'User Defined' 항목을 마우스 우측버튼으로 클릭합니다. 단축메뉴가 나타나면 'New Target Configuration' 항목을 클릭합니다. (미리 저장해둔 Target Configuration 설정이 있을 경우 이를 그대로 사용하셔도 무방합니다. 단, 대상 프로세서는 모듈에 따라, 'TMS320F28P650DK9 혹은 TMS320F28P659DK8-Q1' 로 설정해야 합니다.)

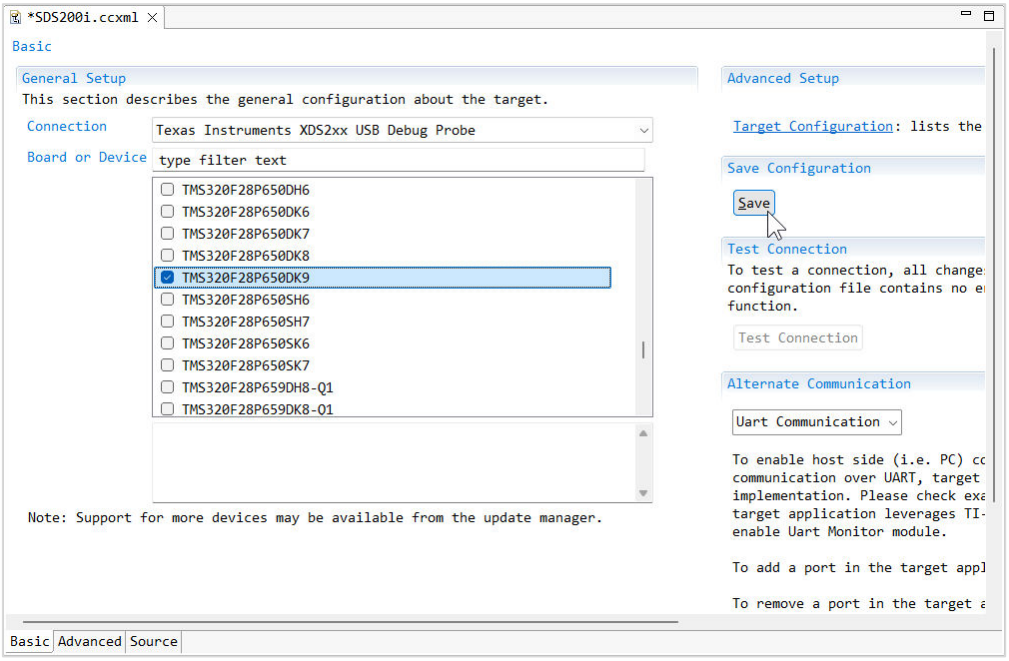


화면에 나타난 'New Target Configuration' 창의 'File name: ' 란에 적당한 이름을 기입합니다. 확장자인 *.ccxml 만 그대로 유지한다면 어떤 이름도 무방합니다. 구분하기 좋은 이름으로 새 타겟환경설정 파일을 생성하세요. 이름을 기입한 후 'Finish' 버튼을 클릭합니다.

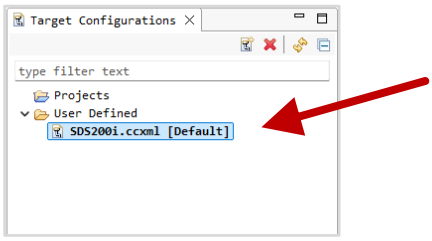


화면 중앙에 생성된 'SDS200i.ccxml' 창(여러분은 전 단계에서 설정하신 이름의 창이 나타날 것 입니다.)에서 'Connection', 'Device' 항목들을 설정합니다. 설정이 끝나면 창 우측의 'Save' 버튼을 클릭하여 변경한 내용을 저장합니다.

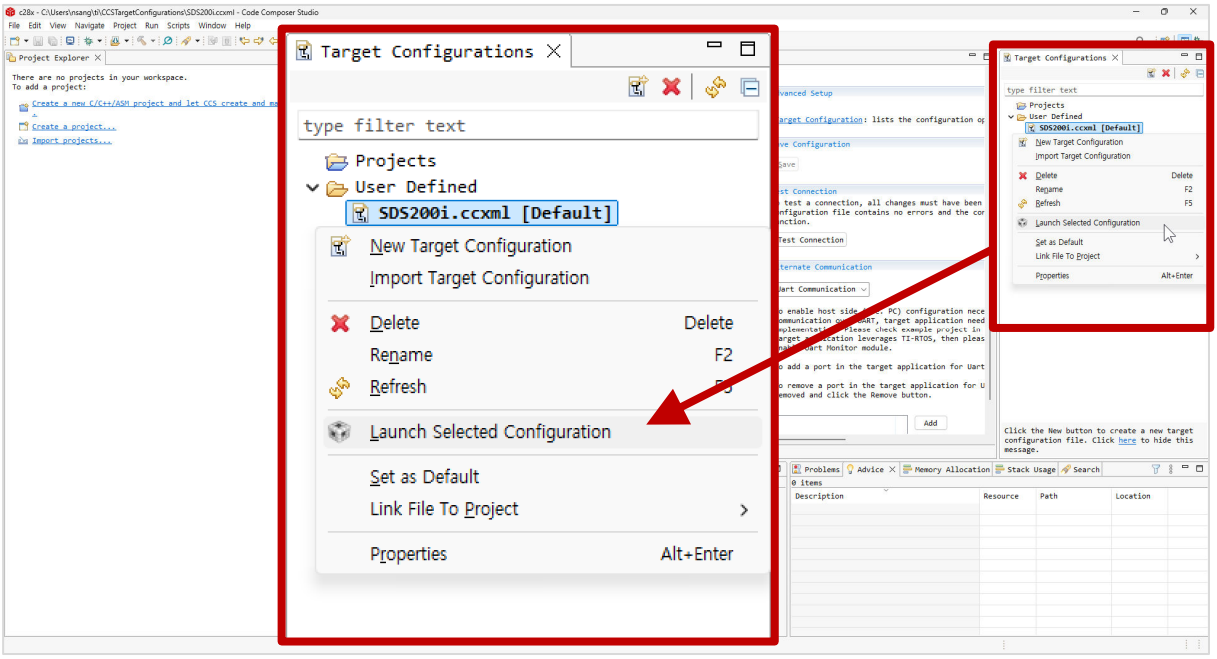
(Device 항목은 모듈에 따라 'TMS320F28P650DK9 혹은 TMS320F28P659DK8-Q1'을 선택하시고, Connection 항목은 사용하시는 JTAG 에뮬레이터에 따라 적합한 드라이버를 선택하시면 됩니다.)

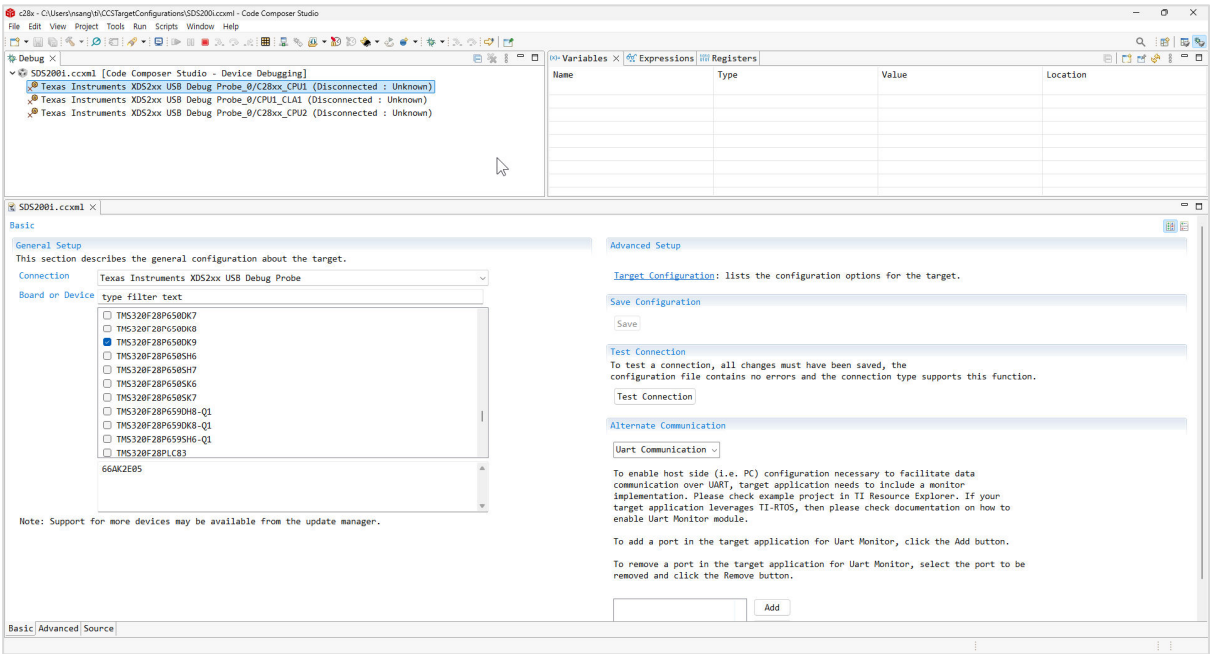


설정 저장을 완료하면 아래의 그림과 같이 CCS 화면 우측에 띄워둔 Target Configurations 창에서 앞서 설정한 타겟 환경설정 파일 'SDS200i.ccxml'를 확인할 수 있습니다.



Target Configurations 창에서 해당 환경설정 파일을 마우스 우측버튼으로 클릭하고 단축메뉴가 나타나면 'Launch Selected Configuration' 항목을 선택합니다. 그렇게 하면 자동으로 CCS Debug 화면구성(Perspective)으로 전환되게 됩니다. (CCS Edit → CCS Debug)


Target Configurations
type filter text
Projects
User Defined
SDS200i.ccxml [Default]
New Target Configuration
Import Target Configuration
Delete
Rename
Refresh
Launch Selected Configuration
Set as Default
Link File To Project
Properties
Alt+Enter


Debug
Variables
Expressions
Registers
Texas Instruments XDS2xx USB Debug Probe_0/C28xx_CPU1 (Disconnected : Unknown)
Texas Instruments XDS2xx USB Debug Probe_0/CPU1_CLA1 (Disconnected : Unknown)
Texas Instruments XDS2xx USB Debug Probe_0/C28xx_CPU2 (Disconnected : Unknown)
SDS200i.ccxml
Basic
General Setup
This section describes the general configuration about the target.
Connection
Texas Instruments XDS2xx USB Debug Probe
Board or Device
type filter text
Advanced Setup
Target Configuration: lists the configuration options for the target.
Save Configuration
Save
Test Connection
To test a connection, all changes must have been saved, the configuration file contains no errors and the connection type supports this function.
Test Connection
Alternate Communication
Uart Communication
Add
Basic
Advanced
Source

5. SMC300 개발보드에 전원을 공급하고, JTAG 에뮬레이터를 Reset 합니다.

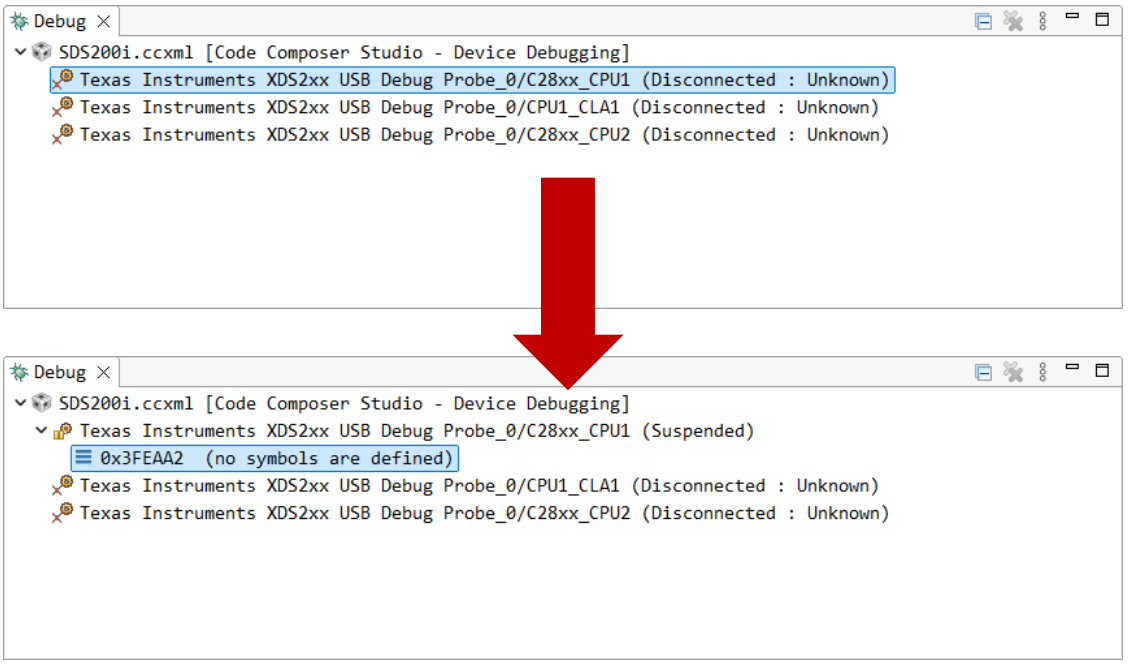
TMS320F28P65x 모듈이 탑재된 SMC300 개발보드에 DC +24V의 전원을 공급하고, PC와 연결된 JTAG 에뮬레이터의 JTAG 커넥터를 개발보드의 JTAG 연결용 핀-헤더(2.54mm 2x7 Male)에 연결합니다. (관련 세부 내용은 매뉴얼 참조)

SMC300 개발보드의 전원 스위치(좌측상단)를 'ON' 위치로 조작하여 보드에 전원을 공급합니다. 단, (SMC300 개발보드의 각종 인터페이스 위치와 조작에 관련한 세부내용은 제품 매뉴얼을 참조하세요.)

CCS 상단메뉴 Run → Reset → Reset Emulator를 선택하여 JTAG 에뮬레이터를 Reset 합니다. 메뉴 항목 클릭 후 화면에 아무런 메시지도 나타나지 않고 변화가 없으면 정상입니다. PC와 JTAG 에뮬레이터 간의 연결에 문제가 있거나, SMC300 개발보드 및 모듈에 전원이 공급되지 않은 경우 CCS 화면 하단의 콘솔(Console) 창에 오류 메시지가 나타납니다.

6. 프로세서와 CCS를 연결(Connect) 합니다.

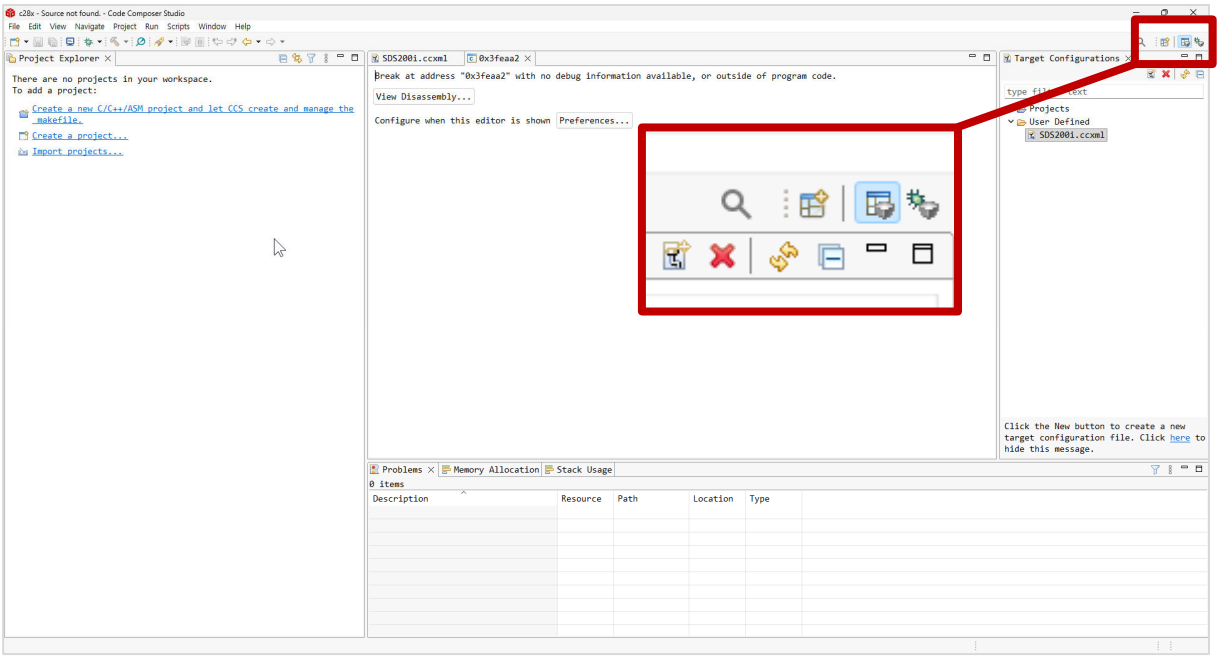
CCS 상단메뉴 Run → Connect Target을 클릭하여 CCS와 TMS320F28P65x 칩의 CPU1을 연결합니다. 정상 연결되면 CCS 화면 좌측 상단의 Debug 창이 아래와 같이 변경됩니다.



7. CCS Edit 화면구성(Perspective)으로 전환하여, 예제 실습을 준비 합니다.

CCS 상단메뉴 Window → Perspective → Open Perspective → CCS Edit 를 선택하거나, CCS 화면 우측 상단의 'CCS Edit' 아이콘을 클릭하여 CCS Edit 화면구성(Perspective)으로 전환합니다. 이제 예제 프로젝트를 불러와서 실습을 진행할 준비가 완료 되었습니다.

위 과정이 정상 진행되면 현재 PC 화면은 아래와 같을 것 입니다. 그렇지 않다면 앞서의 과정 중 빼먹은 것은 없는지 잘 살펴보세요.

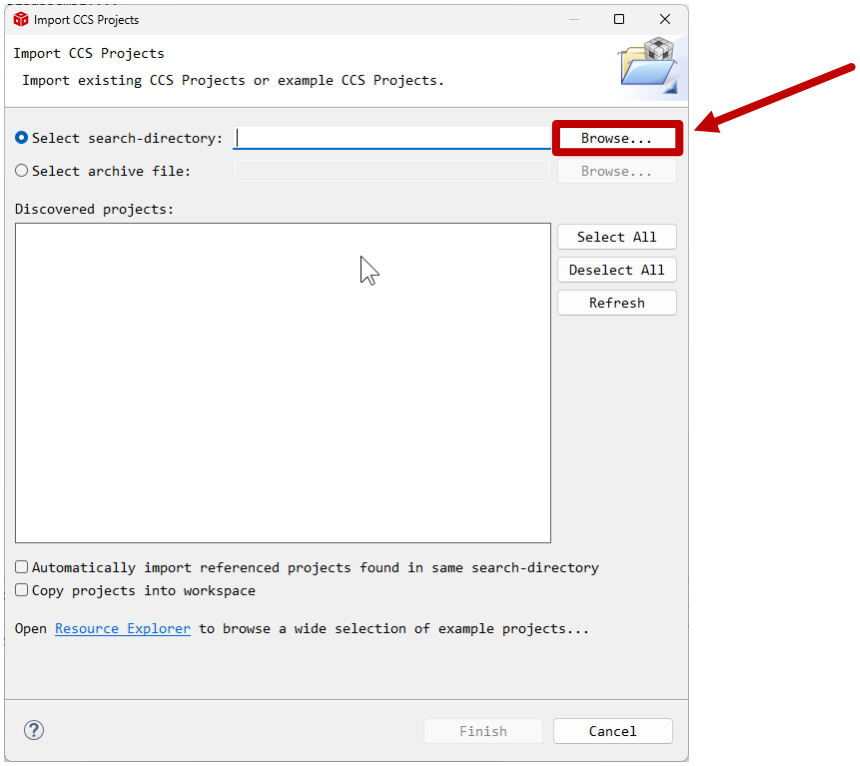


4.2 실습 따라 하기 (Build Level 1)

-------- 이어지는 과정은 CCS Edit 화면구성(Perspective)에서 진행됩니다. ---------

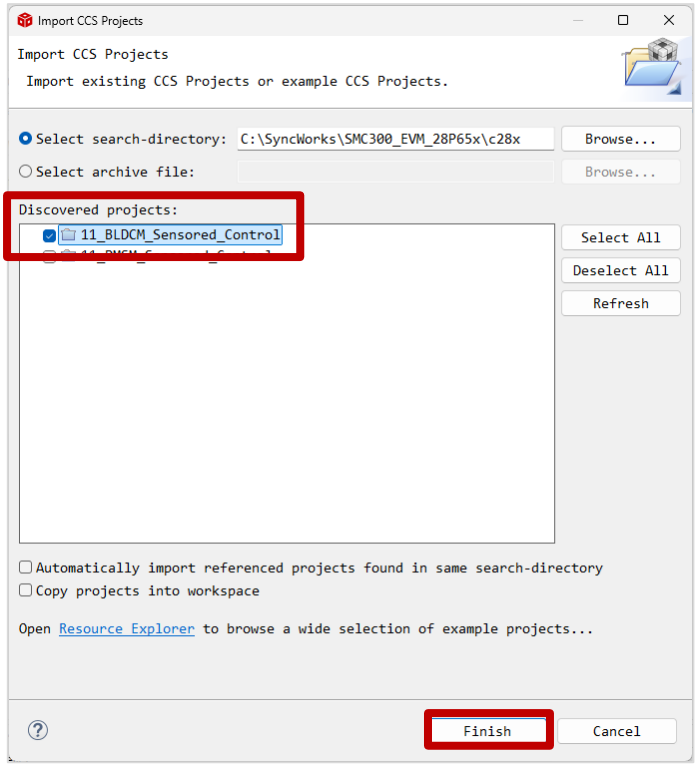
8. 'BLDCM_Sensored_Control' 예제 프로젝트를 불러들입니다. (Open)

CCS 상단메뉴 Project → Import CCS Projects… 를 클릭합니다. 아래의 그림과 같은 창이 나타나면 'Select search-directory: ' 란 옆의 'Browse…' 버튼을 클릭합니다.



디렉터리 탐색 창이 나타나면, Workspace 경로인 C:\SyncWorks\SMC300_EVM_28P65x\c28x 를 선택하고 '확인' 버튼을 클릭합니다.

아래 그림과 같이 'Discovered project: ' 란의 프로젝트 목록에서 'BLDCM_Sensored_Control' 이 선택되었음을 확인(프로젝트 이름 좌측의 체크박스)한 후, 'Finish' 버튼을 클릭합니다.



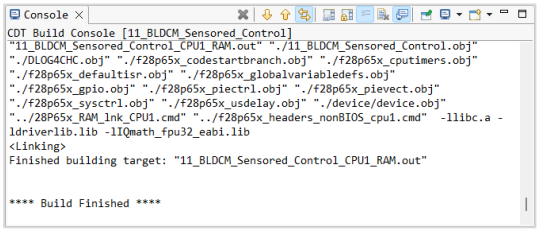
9. 헤더 파일의 BUILDLEVEL 설정을 확인 합니다.

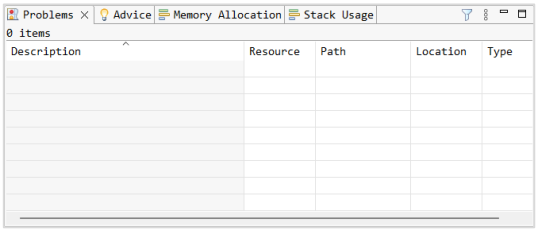
CCS 화면 좌측의 Project Explorer 창에서 '11_BLDCM_Sensored_Control.h' 파일을 찾아 마우스 좌측 버튼으로 더블클릭 합니다. 화면 중앙에 에디터 창을 통해 헤더 파일이 열리면 BUILDLEVEL 항목이 'LEVEL1' 으로 정의되어 있는지 확인합니다.

```
36
37 #define LEVEL0          0       // [내부 테스트용] 홀-센서 신호 확인 (Open-loop)
38 //-------------------------------------------------------------------------------------------------------
39 #define LEVEL1          1       // Open-loop 구동 단계 (Only CW rotation)
40 #define LEVEL2          2       // Open-loop 구동 단계 (CW / CCW rotation)
41 #define LEVEL3          3       // 속도제어 실행 (Closed-loop, CW / CCW rotation, SpeedRef : -0.9 ~ +0.9)
42 //-------------------------------------------------------------------------------------------------------
43
44 #define BUILDLEVEL      LEVEL1
45
```

10. 예제 프로젝트를 Build 합니다.

CCS 화면상단 단축 아이콘들 중  망치모양 아이콘을 클릭하거나, CCS 상단메뉴 Project → Build Project 를 클릭하여 예제 프로젝트를 Build 합니다. Build 과정이 정상적으로 완료되면 아래와 같이 콘솔(Console) 창에 11_BLDCM_Sensored_Control_CPU1_RAM.out 파일이 생성되었음을 알리는 메시지가 나타나고 오류(Problems) 창에 아무런 메시지가 나타나지 않습니다.





11. CCS 화면구성을 CCS Debug로 전환합니다.

이제 Build 한 예제를 칩에 Load 하고 실행시켜보기 위해 CCS 화면구성을 CCS Debug로 전환합니다. CCS 우측 상단의 'CCS Debug' 버튼을 클릭해서 화면 구성을 전환하세요.

-------- 이어지는 과정은 CCS Debug 화면구성(Perspective)에서 진행됩니다. ---------

12. 예제의 Output 파일을 프로세서에 Load 합니다.

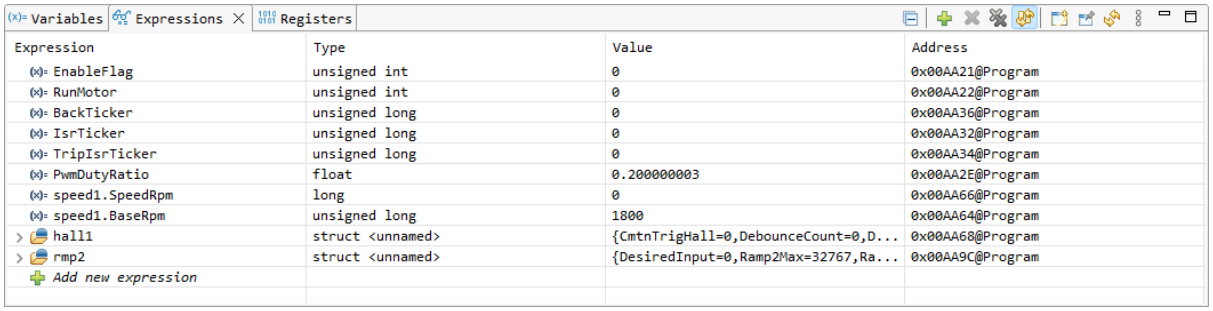
CCS 상단메뉴 Run → Load → Load Program… 을 선택합니다. Load Program 창이 나타나면 창 우측의 'Browse project…' 버튼을 클릭하고 Select a program 창에서 앞서의 과정에서 생성한 '11_BLDCM_Sensored_Control_CPU1_RAM.out' 파일을 선택한 후 'OK' 버튼을 클릭합니다. Select a program 창이 사라지면 Load Program 창의 'OK' 버튼을 클릭해서 Output 파일을 칩에 Load 합니다.

13. Expressions 탭에 관찰 및 조작할 변수들을 등록합니다.

예제를 실행하면서 관찰 및 조작할 변수, 레지스터들을 Expressions 탭에 등록합니다. 이렇게 등록해둔 변수 및 레지스터들은 예제 실행 중 실시간으로 관찰 및 변경이 가능합니다. CCS 화면

우측 상단의 Expressions 탭을 선택한 후 해당 탭의 빈 공간을 마우스 우측 버튼으로 클릭합니다. 단축메뉴가 나타나면 'Import…' 항목을 선택하고 탐색기 창에서 아래 경로의 파일을 선택합니다. (해당 파일은 Build Level 1에서 관찰 및 조작할 변수들을 미리 저장해둔 것 입니다.) 그 외에 살펴보고 싶은 변수나 레지스터들이 있다면 추가로 등록하실 수 있습니다.

C:\SyncWorks\SMC300_EVM_28P65x\c28x\11_BLDCM_Sensored_Control\
Variables_BuildLevel1.txt



Expression	Type	Value	Address
EnableFlag	unsigned int	0	0x00AA21@Program
RunMotor	unsigned int	0	0x00AA22@Program
BackTicker	unsigned long	0	0x00AA36@Program
IsrTicker	unsigned long	0	0x00AA32@Program
TripIsrTicker	unsigned long	0	0x00AA34@Program
PwmDutyRatio	float	0.200000003	0x00AA2E@Program
speed1.SpeedRpm	long	0	0x00AA66@Program
speed1.BaseRpm	unsigned long	1800	0x00AA64@Program
hall1	struct <unnamed>	{CmtnTrigHall=0,DebounceCount=0,D...	0x00AA68@Program
rmp2	struct <unnamed>	{DesiredInput=0,Ramp2Max=32767,Ra...	0x00AA9C@Program
Add new expression			

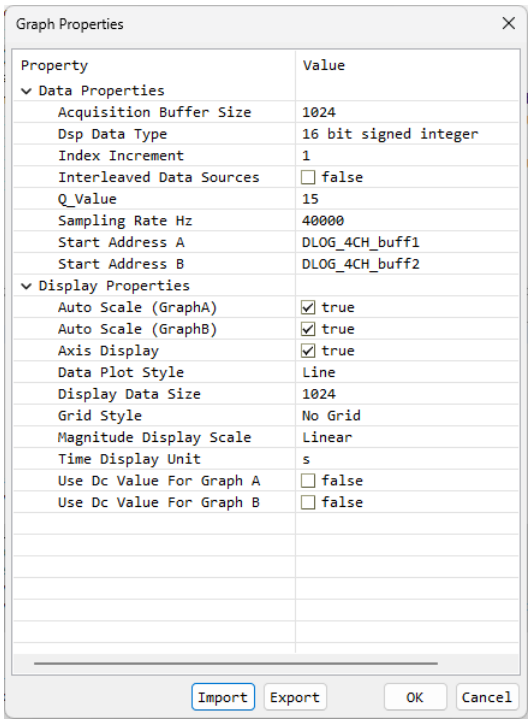
14. Graph 창을 생성합니다.

TI 디지털 모터제어 라이브러리에 포함된 DATA_LOG 모듈이 예제 실행 중 램 영역에 수집한 정보들을 그래프로 관찰하기 위해 아래의 과정을 진행합니다.

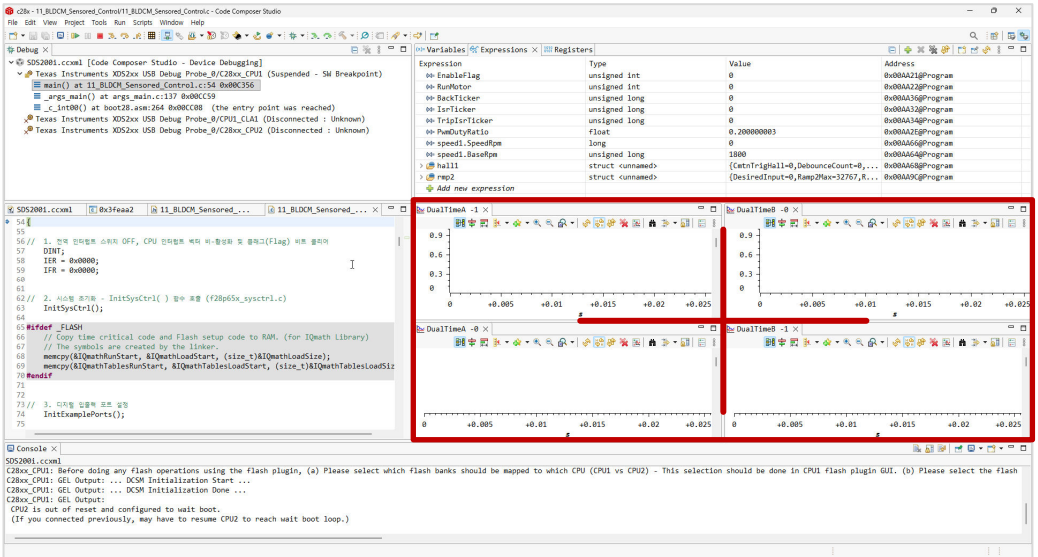
CCS 상단메뉴 Tools → Graph → Dual Time 을 선택하고, 그래프 설정 창 하단의 'Import' 버튼을 클릭합니다. 파일 탐색 창이 나타나면 아래 경로의 파일을 선택합니다.

C:\SyncWorks\SMC300_EVM_28P65x\c28x\11_BLDCM_Sensored_Control\
GraphProperties1.graphProp

위의 파일은 데이터 관찰을 위해 생성할 그래프 창의 각종 설정을 미리 저장해둔 것 입니다. 불러들인 설정이 다음 이미지와 같다면 창 하단의 'OK' 버튼을 클릭해서 그래프 창을 생성합니다.



앞서의 과정을 참고하여, 동일한 경로의 GraphProperties2.graphProp 파일도 Import 해서 추가로 그래프 창을 생성해줍니다. 과정이 모두 완료되면 CCS 화면에 총 4개의 그래프 창이 나타납니다. 생성된 그래프 창은 마우스 왼쪽 버튼으로 창 이름 탭 부분을 클릭 및 드래그해서 원하는 위치로 정렬할 수 있습니다. (아래는 화면 우측 중앙으로 4개의 그래프 창을 정렬한 예입니다.)



15. 프로세서를 Reset 합니다.

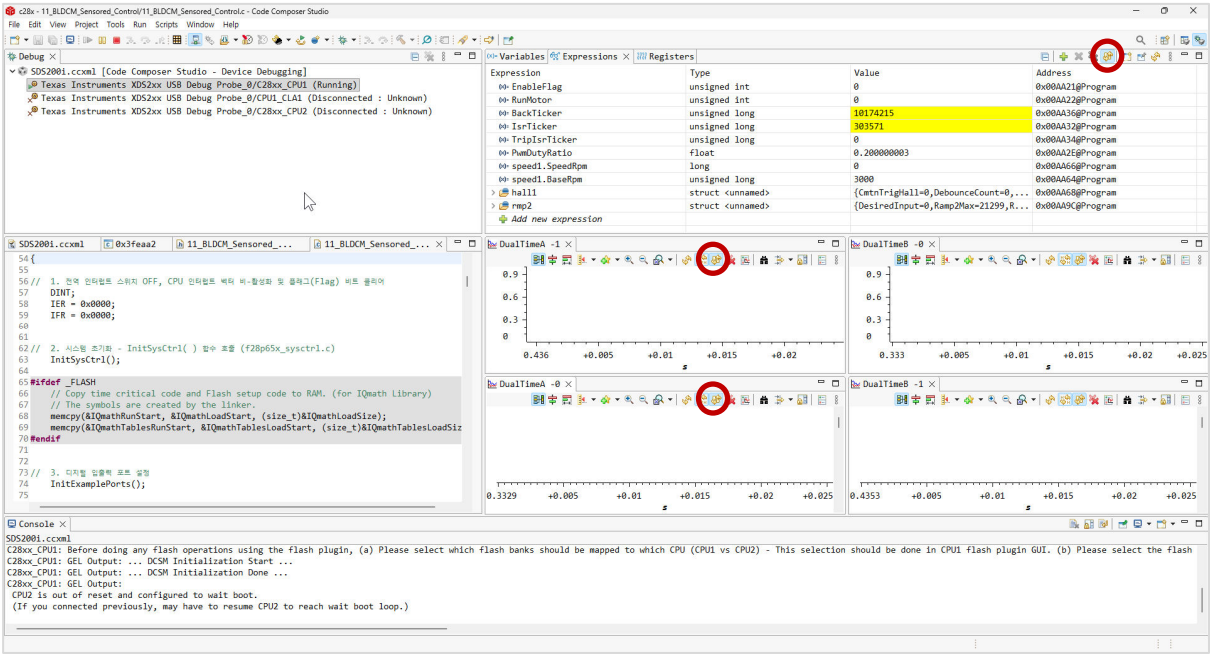
CCS 상단메뉴 Run → Reset → Reset CPU 를 실행합니다. 이어서 Run → Restart 메뉴를 실행합니다. 본 과정을 통해 예제를 실행하기 전 칩의 모든 레지스터들을 깨끗하게 초기 값으로 환원시킬 수 있습니다.

16. 예제 프로그램을 실행합니다.

CCS 상단메뉴 Run → Resume 을 선택하거나, 화면 좌측 상단 Debug 창 상단의 초록색 삼각형 아이콘  을 클릭하여 예제 프로그램을 실행합니다.

17. 앞서 생성한 Graph 창과 Expressions 탭의 실시간 갱신 기능을 켭니다.

Expressions 탭과 Graph 창 우측 상단의  아이콘을 클릭하여, Continuous Refresh 옵션을 켭니다. 이 기능을 통해 예제 실행 중 Expressions 탭에 등록된 변수들과 그래프 창의 내용이 실시간으로 갱신됩니다.



18. SMC300 개발보드의 DC-LINK [A] 전원 스위치를 Ctrl. 위치로 설정합니다.

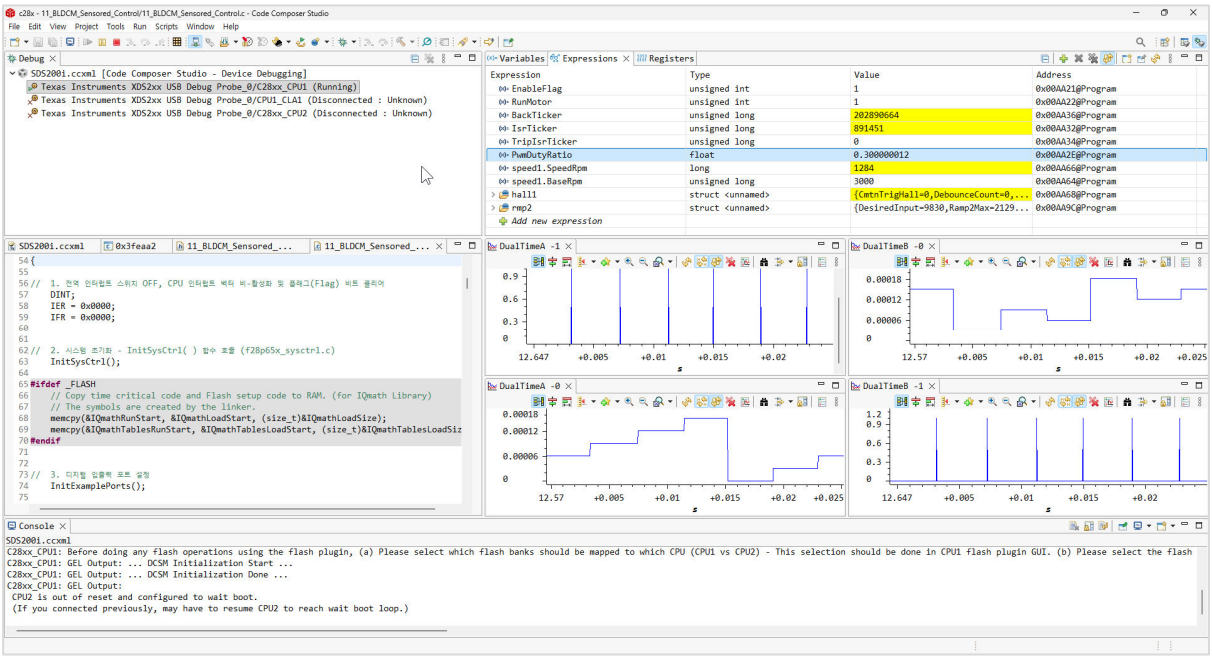
본 예제는 SMC300 개발보드의 모터제어 블록 A를 사용합니다. SMC300 개발보드의 모터제어 블록 A의 DC-LINK 전원 제어용 스위치(SW1400)를 'Ctrl.' 위치로 변경하여 모터제어에 사용될 전원의 ON/OFF를 예제 프로그램에서 제어할 수 있도록 합니다. **(SMC300 EVM V100 버전에 한 함 / SMC300 EVM V110 버전의 경우 물리 스위치가 제거되었으며, eFuse 스위치를 통한 전자제어 방식으로 DC-LINK 전원을 제어합니다.)**

~~**19. SMC300 개발보드의 ADC VREFHI 선택 스위치를 3.3V 위치로 설정합니다.**~~

~~모터제어 블록 A의 ADC VREFHI 선택 스위치 2개를 모두 '3.3V' 위치로 변경하여, 모터제어 블록의 아날로그 신호 가공 회로가 TMS320F28P65x 모듈의 기본 ADC 참조전압 설정과 일치하는 신호를 생성할 수 있도록 합니다.~~ (BLDCM 예제에서 ADC가 사용되지 않으므로 생략 가능)

20. Expressions 탭에 등록되어 있는 'EnableFlag' 변수를 '0' 에서 '1'로 변경합니다.

앞서 Expressions 탭에 등록해둔 주요 변수들 중 'EnableFlag' 변수의 값을 '1'로 변경합니다. 예제 프로그램은 해당 Flag가 1로 변경되면 모터제어에 필요한 알고리즘들이 실행되도록 구성되어 있습니다.



21. 예제 실행결과를 확인하고, 주요 변수들을 조작해봅니다.

Build Level 1은 모터 축의 부하와 관계없이 일정한 Duty-ratio의 PWM 출력을 통해 BLDC 모터를 Open-Loop로 구동하면서 Hall-Effect Sensor의 정보를 정확히 취득하고 있는지, PWM Duty-ratio 변화에 따라 모터의 속력이 달라지는지 확인하는 단계 입니다.

[조작 대상 변수 : PwmDutyRatio]

PwmDutyRatio 변수를 0.0 ~ +0.65 까지의 비율 범위에서 조작해 보실 수 있습니다. 이 변수는 PWM 펄스-폭 지령에 해당되는 비율 변수로 0.0 ~ +1.0의 값이 PWM Duty-ratio 0% ~ 100%와 대응됩니다. 이 지령은 Q15 format으로 가공되어 지연 증감 모듈(rmp2)에 전달되며 값의 변화는 일정한 지연을 통해 점진적으로 반영됩니다. (초기 값 : 0.2 = 20% PWM Duty-ratio)

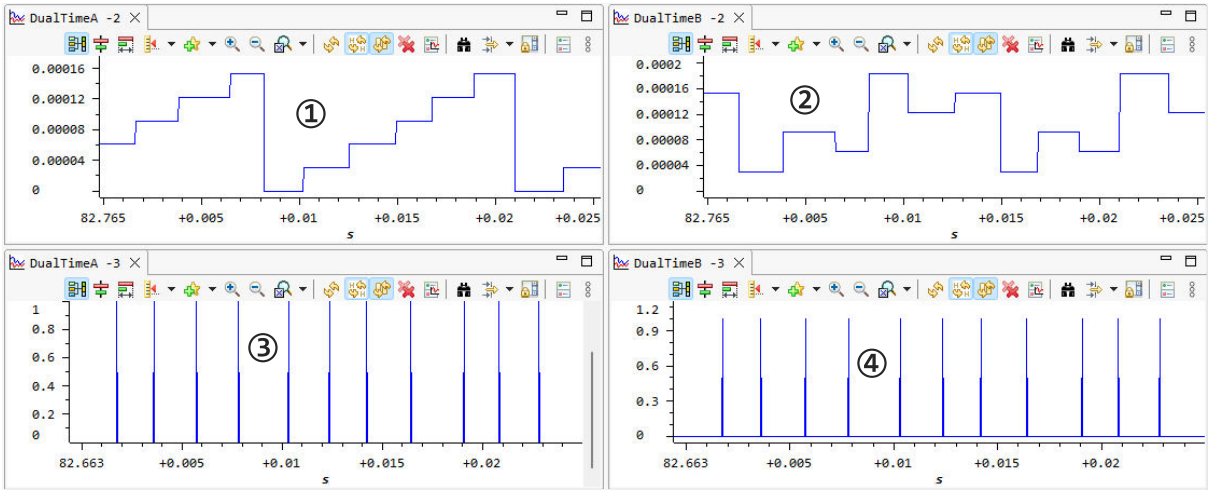
[주요 관찰 대상 변수들]

speed1.SpeedRpm 변수는 현재 모터 속력의 RPM 환산 값을 표시합니다.

hall1 구조체는 모터와 연결된 Hall-Effect Sensor의 신호를 받아 이를 해석하는 모듈 입니다. TI 디지털 모터제어 라이브러리에 포함된 해당 S/W 모듈의 매뉴얼을 참고하여 어떻게 동작하고 있는지 살펴보세요.

Build Level 1에서는 아래와 같은 그래프 창을 확인하실 수 있으며, 아래의 코드를 통해 DATA_LOG 모듈이 어떤 변수들의 변화이력을 누적하고 있는지 알 수 있습니다. (DATA_LOG 모듈의 1~4 채널 ➔ 그래프 창 1~4)

```
365     //  Connect inputs of the DATALOG module
366     DlogCh1 = (int16)mod1.Counter;
367     DlogCh2 = hall1.HallGpioAccepted;
368     DlogCh3 = (int16)mod1.TrigInput;
369     DlogCh4 = hall1.CmtnTrigHall;
```



22. 안전하게 예제 프로그램의 실행을 중지합니다.

아래의 순서에 따라 안전하게 예제의 실행을 중지합니다.

a). Expressions 탭에 등록된 'EnableFlag' 변수를 '0'으로 변경합니다. (변경 후 모터가 정지하는 것을 확인합니다.)

b). SMC300 개발보드의 모터제어 블록 A – DC-LINK [A] 전원 스위치(SW1400)를 'OFF' 위치로 변경하면 하드웨어적으로 모터 구동용 전원을 차단할 수 있습니다. (SMC300 EVM V100에 한 함)

c). CCS 상단메뉴 Run → Suspend 를 클릭하여 프로그램 실행을 중지합니다. (Halt Program)

d). CCS를 종료하고 SMC300 개발보드의 통합 전원 스위치(PSW1000)를 'OFF' 위치로 변경하여 보드에 공급된 전원을 차단합니다.

e). 전원 스위치를 OFF로 조작 후, 수 초간 기다렸다가 JTAG 에뮬레이터 및 프로세서 모듈을 보드와 분리합니다.

4.3 실습 따라 하기 (Build Level 2)

예제 프로젝트를 Build 하고, 실행하기까지의 과정은 Build Level 1 실습 때와 동일합니다. 앞서 설명된 내용과 아래를 참고하여 예제 프로젝트를 Build Level 2(헤더 파일의 정의 구문 수정)로 설정하여 실행합니다.

Build Level 수정 : 11_BLDCM_Sensored_Control.h 파일의 'BUILDLEVEL' 값 변경 (LEVEL2)

```
#define LEVEL0          0       // [내부 테스트용] 홀-센서 신호 확인 (Open-loop)
//-----------------------------------------------------------------------------------------------
#define LEVEL1          1       // Open-loop 구동 단계 (Only CW rotation)
#define LEVEL2          2       // Open-loop 구동 단계 (CW / CCW rotation)
#define LEVEL3          3       // 속도제어 실행 (Closed-loop, CW / CCW rotation, SpeedRef : -0.9 ~ +0.9)
//-----------------------------------------------------------------------------------------------

#define BUILDLEVEL      LEVEL2
```

Expressions 탭 변수등록용 파일 : Variables_BuildLevel2.txt

Graph 창 생성용 파일 1 : GraphProperties1.graphProp

Graph 창 생성용 파일 2 : GraphProperties2.graphProp

예제 프로그램을 Build 하고 실행해서 Expressions 탭에 등록된 'EnableFlag'를 '1'로 변경하세요.

-------- 이어지는 과정은 CCS Debug 화면구성(Perspective)에서 진행됩니다. ---------

-------- 예제 실행 명령(Run → Resume) 후의 과정 안내입니다. ---------

23. 예제 실행결과를 확인하고, 주요 변수들을 조작해봅니다.

Build Level 2는 앞서의 Build Level 1 코드에 모터의 회전방향을 변경할 수 있는 코드를 추가한 예 입니다. PwmDutyRatio 값 변화에 따라 모터가 시계방향(CW, 0) 혹은 반-시계방향(CCW, 1)으로 회전합니다.

[조작 대상 변수 : PwmDutyratio]

PwmDutyratio 변수를 -0.65 ~ +0.65 까지의 비율 범위에서 조작해보실 수 있습니다. 이 변수는 PWM 펄스-폭 지령에 해당되며 부호는 회전방향에 반영됩니다. Expressions 탭에 등록된 mod1.Direction 변수의 값을 통해 회전 방향을 확인할 수 있습니다. (0: CW / 1: CCW)

[주요 관찰 대상 변수들]

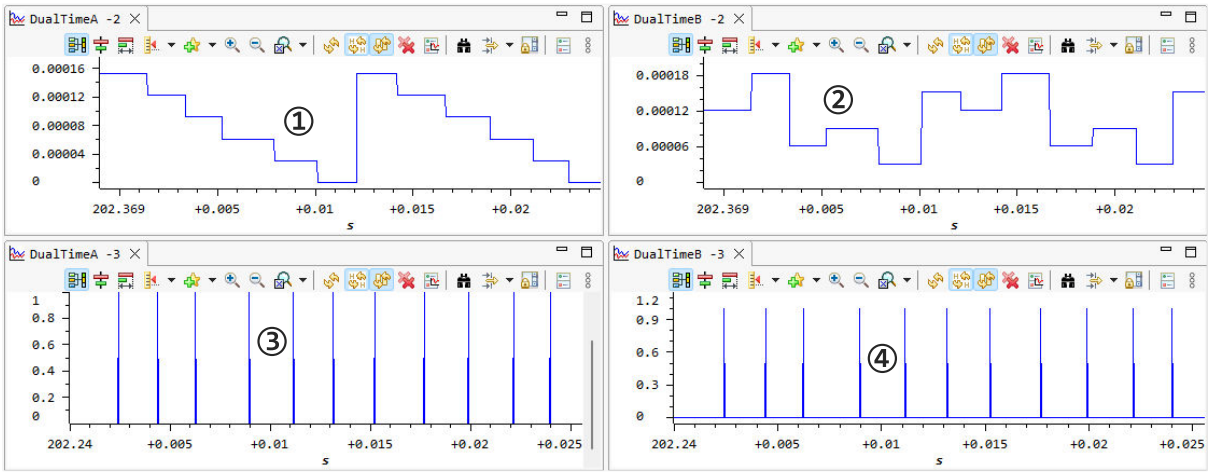
speed1.SpeedRpm 변수는 현재 모터 속력의 RPM 환산 값을 표시합니다.

hall1 구조체는 모터와 연결된 Hall-Effect Sensor의 신호를 받아 이를 해석하는 모듈 입니다.

PwmDutyRatio 지령 값은 Global Q format으로 변경되어 지연 증감 제어모듈(rc1)에 전달되고, PWM 펄스-폭 및 회전방향에 반영되도록 구성되어 있습니다. (Global Q Default: Q24)

Build Level 2에서는 아래와 같은 그래프 창을 확인하실 수 있으며, 아래의 코드를 통해 DATA_LOG 모듈이 어떤 변수들의 변화이력을 누적하고 있는지 알 수 있습니다. (DATA_LOG 모듈의 1~4 채널 ➔ 그래프 창 1~4)

```
552     //  Connect inputs of the DATALOG module
553     DlogCh1 = (int16)mod1.Counter;
554     DlogCh2 = hall1.HallGpioAccepted;
555     DlogCh3 = (int16)mod1.TrigInput;
556     DlogCh4 = hall1.CmtnTrigHall;
```



24. 안전하게 예제 프로그램의 실행을 중지합니다.

앞서 설명된 절차에 따라 안전하게 예제의 실행을 중지합니다.

4.4 실습 따라 하기 (Build Level 3)

예제 프로젝트를 Build 하고, 실행하기까지의 과정은 Build Level 1, 2 실습 때와 동일합니다. 앞서 설명된 내용과 아래를 참고하여 예제 프로젝트를 Build Level 3(헤더 파일의 정의 구문 수정)으로 설정하여 실행합니다.

Build Level 수정 : 11_BLDCM_Sensored_Control.h 파일의 'BUILDLEVEL' 값 변경 (LEVEL3)

```
#define LEVEL0          0       // [내부 테스트용] 홀-센서 신호 확인 (Open-loop)
//-----------------------------------------------------------------------------------------------
#define LEVEL1          1       // Open-loop 구동 단계 (Only CW rotation)
#define LEVEL2          2       // Open-loop 구동 단계 (CW / CCW rotation)
#define LEVEL3          3       // 속도제어 실행 (Closed-loop, CW / CCW rotation, SpeedRef : -0.9 ~ +0.9)
//-----------------------------------------------------------------------------------------------

#define BUILDLEVEL      LEVEL3
```

Expressions 탭 변수등록용 파일 : Variables_BuildLevel3.txt

Graph 창 생성용 파일 1 : GraphProperties1.graphProp

Graph 창 생성용 파일 2 : GraphProperties2.graphProp

예제 프로그램을 Build 하고 실행해서 Expressions 탭에 등록된 'EnableFlag'를 '1'로 변경하세요.

-------- 이어지는 과정은 CCS Debug 화면구성(Perspective)에서 진행됩니다. ---------

-------- 예제 실행 명령(Run → Resume) 후의 과정 안내입니다. ---------

25. 예제 실행결과를 확인하고, 주요 변수들을 조작해봅니다.

Build Level 3은 PI 제어 모듈을 통해 모터 축에 걸리는 부하의 양에 따라 PWM 출력 펄스의 Duty-ratio를 적절히 제어하는 Speed Closed-Loop 단계 입니다. 'hall1' S/W 모듈이 Hall-Effect Sensor로부터 정보를 얻어내고 'speed1' 모듈이 현재 모터 회전축의 속도를 산출해냅니다. 계산된 모터의 속도는 PI 제어 모듈의 Feedback으로 활용되어 사용자의 속도 지령과의 오차 및 미리 설정된 계수에 의해 PWM 펄스의 Duty-ratio를 제어하게 됩니다. (예제의 PI 제어기 계수는 최적화된 값이 아니며, 안전한 구동이 가능하도록 작은 값으로 초기화 되어 있습니다. 이 계수들은 추후 실제 모터제어 시스템을 개발하실 때 구동할 모터나 부하에 따라 적절히 수정되어야 할 부분 입니다.)

[속도 지령 값 변경 방법]

예제에서 PI 제어기의 Reference 값으로 사용되는 사용자 속도 지령은 'SpeedRef' 변수 입니다. -1.0 ~ +1.0 사이의 비율 값으로 모터의 속도 지령을 나타내며, 예제에서 SpeedRef 값이 0 일 경우, 이는 RPM으로 환산했을 때 0 RPM의 속도지령을 준 것과 동일하고 SpeedRef 1.0 은 3,000RPM의 속도지령과 동일합니다. 아울러, SpeedRef 변수의 부호는 모터의 회전방향을 의미합니다. SpeedRef 값이 + 인 경우 시계방향(CW) 회전에 대한 속도지령을 의미하며, - 인 경우 시계반대방향(CCW) 회전에 대한 속도지령이 됩니다. 예제에서는 SpeedRef 값을 -0.9 ~ +0.9 사이로 제한하고 있습니다. (예, SpeedRef -0.5는 속도지령 -1,500RPM을 의미합니다.)

예제를 실행하고 적절한 SpeedRef 값 설정 후, 모터의 회전축을 손으로 살짝 잡아 부하를 가해보시면 PI 제어기에 따라 PWM Duty-ratio가 커지고 잠시 후 모터가 사용자 속도지령 값을 추종하도록 제어됨을 확인하실 수 있습니다.

[속도 PI 제어기 계수 변경 방법]

예제에 적용된 PI 제어기의 비례(Proportional Gain), 적분(Integral Gain) 계수를 변경하려면 'SpeedKp', 'SpeedKi' 변수의 값을 수정하시면 됩니다.

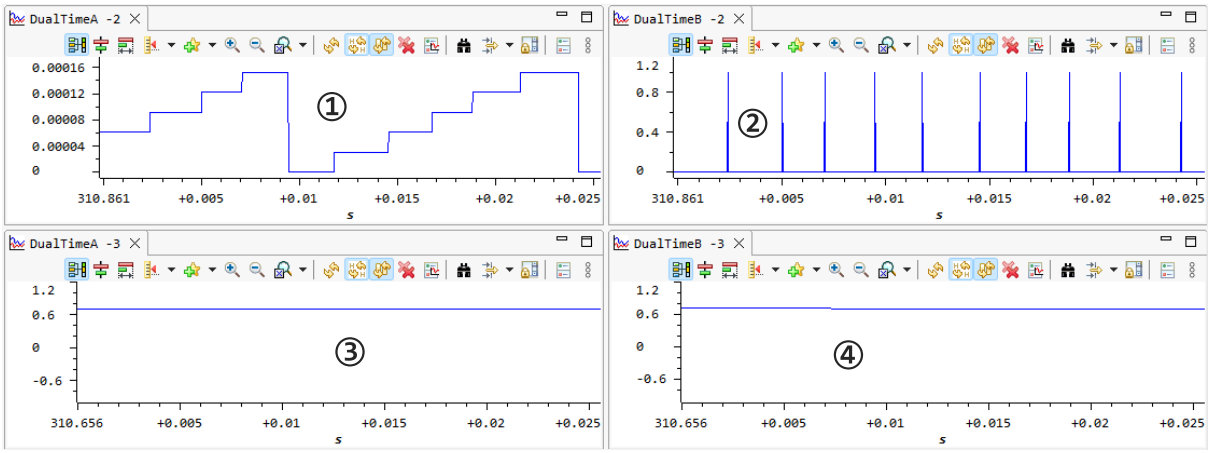
Graph 창을 통해 모터에 가해지는 부하의 크기나 유무에 따라, 사용자 속도 지령 추종 특성을 관찰하시려면 Expressions 탭에 등록된 dlog.prescalar 변수의 값을 '1' 이 아닌 '500' 정도로 변경해보세요. DATA_LOG 함수가 매 인터럽트 주기마다 누적하던 데이터를 인터럽트 주기 500회 마다 한 번씩 누적함으로써 보다 긴 시간 범위에서의 변화 이력을 관찰하기 용이하게 됩니다. (dlog.prescalar 변수 값을 보다 큰 값으로 변경하는 것은 계측의 시간 범위를 보다 길게 변경하는 것과 같습니다.)

[주요 관찰 대상 변수]

speed1.SpeedRpm 변수는 현재의 모터 회전 축 속도를 RPM 단위로 표시합니다. SpeedRef 변수의 설정 값과 PI 제어 모듈(pid1_spd)에 의해 제어되고 있는 현재의 모터 속도를 비교해보세요. 손으로 회전축을 살짝 잡아 부하를 주고, PI 제어기의 계수 값들도 변경해보면서 모터의 회전/제어 특성이 어떻게 달라지는지도 확인해보세요.

Build Level 3에서는 아래와 같은 그래프 창을 확인하실 수 있으며, 아래의 코드를 통해 DATA_LOG 모듈이 어떤 변수들의 변화이력을 누적하고 있는지 알 수 있습니다. (DATA_LOG 모듈의 1~4 채널 ➔ 그래프 창 1~4)

```
743     //  Connect inputs of the DATALOG module
744     DlogCh1 = pwm1.CmtnPointer;
745     DlogCh2 = hall1.CmtnTrigHall;
746     DlogCh3 = (int16)_IQtoIQ15(pid1_spd.Ref);
747     DlogCh4 = (int16)_IQtoIQ15(pid1_spd.Fdb);
```



26. 안전하게 예제 프로그램의 실행을 중지합니다.

앞서 설명된 절차에 따라 안전하게 예제의 실행을 중지합니다.